Homework 1
Running a Large Language Model Locally

Deploying Qwen with Ollama and Building a ChatGPT-Style Interface

1 Three Homework assignments with a common purpose.

The objective o the three homework assignments is to build a conversational agent, powered with a local LLM, that has two major characteristics, a very specific personality and a database knowledge that it is additional to the LLM trained content.

In this way you can reuse your learnings week to week and build on top of the learnings of the previous week.

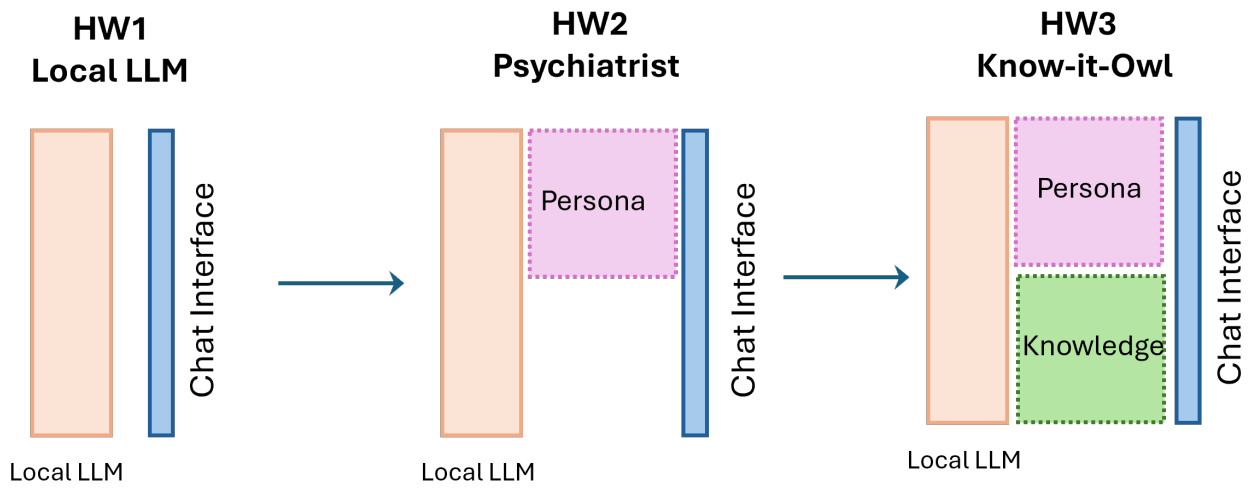

Figure 1: An evolution set of homework assignments

The homework is technically complex, but you can (must) use advanced development techniques based on AI.

1.1 How to Use Claude Code or similar assistants to Learn While Building

AI coding assistants like **Claude Code** can be a tremendous accelerator — but used carelessly, they can also short-circuit your learning entirely. The difference lies in *how* you interact with them. This section gives you a workflow that lets you move fast while still genuinely understanding what you build.

1.2 The Core Principle: Drive, Don't Passenger

Think of Claude Code as a very knowledgeable co-pilot who can type faster than you. The moment you hand over the wheel entirely — pasting a prompt, accepting the output, and moving on — you stop learning. The goal is to stay in the driver's seat: you decide *what* to build and *why*, and you use the assistant to help you figure out *how*.

A useful rule of thumb: **never accept code you cannot read aloud and explain line by line.**

1.3 A Recommended Workflow

1. **Attempt it yourself first.** Before opening Claude Code, spend 15–20 minutes trying to solve the problem on your own. Even if you get stuck, this primes your brain with the right questions and makes the assistant's answer far more meaningful.
2. **Ask for explanation alongside code.** Always append *"and explain each part"* or *"walk me through the logic"* to your prompts. Reading an explanation forces you to check your understanding and spot gaps.
3. **Ask "why", not just "what".** After receiving code, follow up with questions like *"Why did you use a generator here instead of returning a list?"* or *"What would break if I removed the system prompt from the history?"*. These follow-up questions are where the real learning happens.
4. **Break it deliberately.** Once you have working code, modify something — remove a line, change a parameter, introduce a small bug — and predict what will happen before running it. Then run it and see if you were right. This is one of the most effective ways to build intuition.
5. **Re-implement from memory.** After understanding a piece of code, close it and rewrite it yourself from scratch. You do not need to reproduce it perfectly — the act of reconstructing it consolidates your understanding far better than re-reading it.
6. **Use Claude to review your own code.** Once you have written something yourself, ask Claude Code to critique it: *"What are the weaknesses of this implementation?"* or *"How would a senior engineer improve this?"*. This gives you targeted feedback without replacing your creative effort.

1.4 Prompts That Teach vs. Prompts That Replace

The way you phrase a prompt determines whether you learn or just receive an answer.

Prompts that replace learning	Prompts that build understanding
"Write me the full app."	"I have written this function — what is wrong with it and why?"
"Give me the code for streaming output."	"Explain how streaming works in the Ollama API, then show me the minimal code needed."
"Fix my bug."	"Here is my code and the error. What concept am I misunderstanding that caused this?"
"Add a reset button to my UI."	"What state needs to be cleared when a conversation resets? Help me think through it before writing code."

1.5 A Honest Note on Effort and Grades

It is entirely possible to complete this homework in under an hour using Claude Code with minimal understanding. It is also entirely possible to spend three hours with it and come away with a solid grasp of local LLM inference, Python application structure, and streaming APIs. **The tool does not determine the outcome — your intellectual engagement does.**

The reflection document you submit, and any follow-up questions during evaluation, are specifically designed to surface whether you understood what you built. An application that works perfectly but that you cannot explain will not receive full marks.

Use the assistant. Learn from it. Own what you submit.

2 Learning Objectives

In this individual homework, you will install and run a **state-of-the-art Large Language Model entirely on your own machine**, with no data ever leaving your computer. Using **Qwen** (Alibaba's open-weight model family) served locally via **Ollama**, and a Python backend, you will build a fully functional chat interface that looks and feels like ChatGPT.

The goals of this assignment are:

- Understand what it means to run an LLM **locally** and how it differs from using a cloud API.
- Install and configure **Ollama** as a local model server.
- Download and run the latest **Qwen** model family.
- Build a **chat interface** using **Streamlit** that supports multi-turn conversations.

- Understand and articulate the **privacy and data sovereignty** advantages of local inference.

The final objective will be a chatbot that has personality or simulates a Persona and uses its own knowledge provided by a large database that will be accesses using RAG (Retrieval Augmented Generation).

The assignments are fun, and a bit complex, but remember you can use AI to develop the applications.

3 Motivation: Why Run an LLM Locally?

Before diving into the technical steps, it is important to understand *why* running a model locally matters — not just as a technical exercise, but as a principled decision.

3.1 Privacy and Data Sovereignty

When you send a message to ChatGPT, Claude, or Gemini, your text travels over the internet to a remote server, is processed there, and the provider *may* log, store, or use that data for future training. This is a significant concern in many real-world scenarios:

- **Medical and legal contexts:** A doctor querying symptoms or a lawyer summarizing a confidential case brief cannot safely use a cloud LLM without violating patient/client confidentiality obligations (e.g., HIPAA, GDPR).
- **Corporate intellectual property:** Employees pasting internal strategy documents, source code, or financial data into a cloud chatbot may inadvertently expose trade secrets.
- **Sensitive personal data:** Conversations about personal finances, mental health, or family situations may expose users to profiling or data breaches.
- **Regulated industries:** Banking, defence, and government sectors often prohibit sending data to third-party cloud services entirely.

Running a model locally solves all of these problems. Your prompts never leave your machine. There is no API call, no server log, no third-party data processor. The model runs entirely on your CPU/GPU, and the conversation disappears when the session ends — unless you explicitly save it yourself.

3.2 Cost and Independence

Local models also eliminate per-token API costs and remove dependency on a vendor's availability, rate limits, or pricing changes. Once the model is downloaded, inference is free.

4 What is Qwen?

Qwen (short for *Qianwen*, meaning "thousand questions" in Chinese) is a family of open-weight LLMs developed by **Alibaba Cloud**. As of 2025–2026, Qwen has become one of the top-performing open model families, competing directly with Meta's LLaMA and Google's Gemma.

Model	Parameters	Context Window	Best For
`qwen3:latest`	5.2GB	40K tokens	standard 3 model
`qwen2.5:0.5b`	0.5B	32K tokens	Laptops without GPU
`qwen2.5:7b`	7B	128K tokens	**Recommended for this HW**
`qwen2.5:14b`	14B	128K tokens	Laptops with good GPU
`qwen3:32b`	20GB	40KK tokens	DGX

Recommendation: Use `qwen2.5:7b` as your default. It runs on most modern laptops (even without a dedicated GPU, using CPU inference) and delivers very strong conversational quality. If your machine struggles, fall back to `qwen2.5:3b`.

Key strengths of Qwen 2.5:

- Strong multilingual support (English, Spanish, Chinese, Arabic, and many more)
- Excellent instruction-following and chat capabilities
- Long context window (128K tokens) — can process entire documents
- Fully open weights under the Apache 2.0 licence

5 Architecture Overview

The system you will build has three layers, as shown in Figure 2.

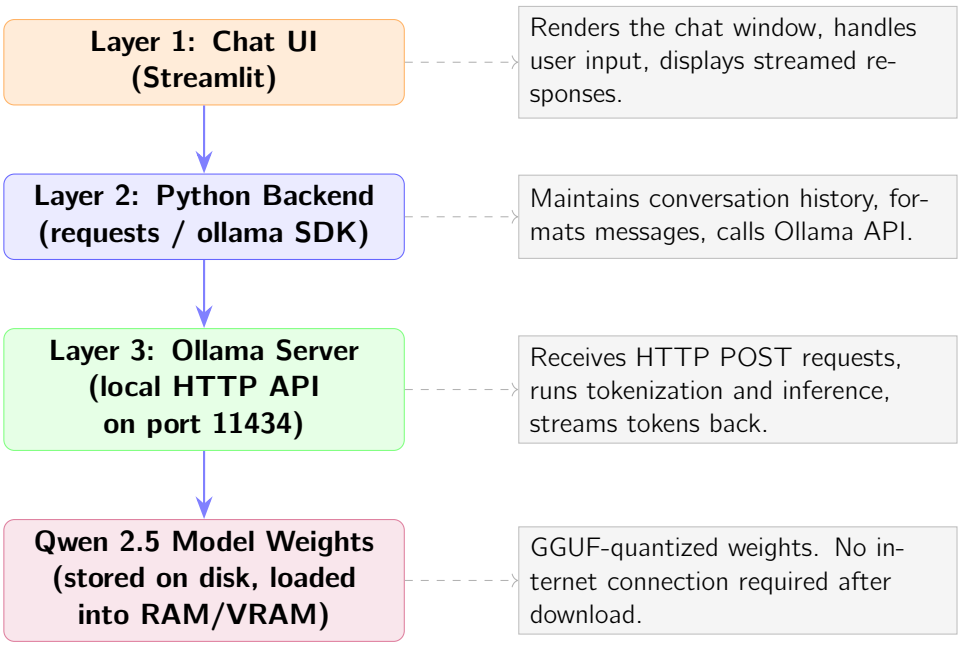

Figure 2: Three-layer architecture of the local chat system.

The key insight is that **all three layers run on your machine**. There is no external network call after the initial model download.

6 Step-by-Step Setup

6.1 Step 1 — Install Ollama

Ollama is an open-source tool that makes running LLMs locally as simple as running a Docker container. It handles model downloading, quantization, hardware detection (CPU vs GPU), and exposes a local HTTP API.

Installation:

```
# macOS
brew install ollama

# Linux
curl -fsSL https://ollama.com/install.sh | sh

# Windows
# Download the installer from: https://ollama.com/download/windows
# Run the .exe and follow the wizard
```

After installation, verify it is running:

```
ollama --version
# Expected output: ollama version 0.x.x
```

6.2 Step 2 — Download Qwen

```
# Recommended for most students (requires ~5GB disk space)
ollama pull qwen2.5:7b

# Lighter alternative for low-memory machines
ollama pull qwen2.5:3b

# Check downloaded models
ollama list
```

The download may take a few minutes depending on your connection. The model is stored in `~/.ollama/models/` and will persist across reboots.

Quick sanity check — chat in the terminal:

```
ollama run qwen2.5:7b
# Type: "Hello, what are you?" and verify you get a response
# Press Ctrl+D to exit
```

6.3 Step 3 — Understand the Ollama API

Once Ollama is running, it exposes a local REST API at `http://localhost:11434`. The key endpoint is:

```
POST http://localhost:11434/api/chat
```

You can test it directly with `curl`:

```
curl http://localhost:11434/api/chat \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen2.5:7b",
    "messages": [
      {"role": "user", "content": "What is the capital of France?"}
    ],
    "stream": false
  }'
```

Understanding this API is important: your Python backend will call exactly this endpoint.

6.4 Step 4 — Install Python Dependencies

```
pip install ollama streamlit requests
```

7 Implementation Guide

7.1 Part A — Backend: Conversation Manager

The backbone of your chat application is a Python class that maintains the conversation history and communicates with Ollama. This is what makes the chat *multi-turn*: the model remembers previous exchanges.

```python
import ollama

class ConversationManager:
    def __init__(self, model: str = "qwen2.5:7b", system_prompt: str = None):
        self.model = model
        self.history = []
        if system_prompt:
            self.history.append({
                "role": "system",
                "content": system_prompt
            })
```

```python
    def chat(self, user_message: str) -> str:
        """Send a message and get a response. History is maintained."""
        self.history.append({
            "role": "user",
            "content": user_message
        })

        response = ollama.chat(
            model=self.model,
            messages=self.history
        )

        assistant_message = response["message"]["content"]
        self.history.append({
            "role": "assistant",
            "content": assistant_message
        })
        return assistant_message

    def stream_chat(self, user_message: str):
        """Generator version: yields tokens as they are produced."""
        self.history.append({
            "role": "user",
            "content": user_message
        })

        full_response = ""
        for chunk in ollama.chat(
            model=self.model,
            messages=self.history,
            stream=True
        ):
            token = chunk["message"]["content"]
            full_response += token
            yield token # Yield each token as it arrives

        self.history.append({
            "role": "assistant",
            "content": full_response
        })

    def reset(self):
        """Clear conversation history (keep system prompt if set)."""
        self.history = [m for m in self.history if m["role"] == "system"]
```

Key concept — conversation history: Notice that every call to `ollama.chat()` includes the *entire* message history. This is how LLMs maintain context: they do not have internal memory between calls.

You are responsible for sending the full conversation each time. This is identical to how the ChatGPT API works.

7.2 Part B — Frontend: Streamlit

Streamlit is the required framework for building your chat interface. It produces a clean, functional UI with minimal boilerplate and integrates naturally with Python backends.

```
import streamlit as st
import ollama

st.set_page_config(page_title="Qwen Local Chat", page_icon="")
st.title("Qwen 2.5 - Local Chat")
st.caption("Running entirely on your machine. No data leaves this device.")

# Initialise session state
if "messages" not in st.session_state:
    st.session_state.messages = [
        {"role": "system",
         "content": "You are a helpful, concise, and friendly assistant."}
    ]

# Render existing messages
for msg in st.session_state.messages:
    if msg["role"] != "system":
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

# Handle new user input
if prompt := st.chat_input("Type your message..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    with st.chat_message("assistant"):
        placeholder = st.empty()
        full_response = ""

        # Stream response token by token
        for chunk in ollama.chat(
            model="qwen2.5:7b",
            messages=st.session_state.messages,
            stream=True
        ):
            full_response += chunk["message"]["content"]
            placeholder.markdown(full_response + "") # Blinking cursor effect

        placeholder.markdown(full_response)
```

```
    st.session_state.messages.append(
        {"role": "assistant", "content": full_response}
    )

# Sidebar controls
with st.sidebar:
    st.header("Settings")
    st.caption("Model: qwen2.5:7b")
    if st.button("Clear conversation"):
        st.session_state.messages = [st.session_state.messages[0]]
        st.rerun()
```

Run it with:

```
streamlit run app.py
# Opens automatically at http://localhost:8501
```

8 Required Features

Your submitted application **must** implement all of the following:

1. **Multi-turn conversation:** The model must remember previous messages in the session. Sending only the latest message is not acceptable.
2. **Streaming output:** Responses must appear token-by-token, not all at once after a delay.
3. **Conversation reset:** A button to clear history and start fresh.
4. **System prompt:** A configurable system prompt that sets the assistant's personality or role (can be hardcoded or exposed in the UI).
5. **Model indicator:** The UI must display which model is currently running (e.g., `qwen2.5:7b`).

9 Additional features you may add

If you completed the base requirements and wish to go further may implement any of the following:

- **Model selector:** A dropdown in the UI to switch between `qwen2.5:3b`, `qwen2.5:7b`, and `qwen2.5:14b` at runtime.
- **Conversation export:** A button that saves the current chat history to a `.txt` or `.json` file.
- **System prompt editor:** An editable text box in the UI that lets the user change the system prompt before starting a conversation (e.g., "You are a Python tutor").
- **Token counter:** Display the number of tokens used in the current conversation (input and response

- **Temperature control:** A slider to adjust the model's `temperature` parameter (0 = deterministic, 1 = creative).

10 Hello World program with Ollama

Before you start use this program (Notebook) to see if everything is in place

```
https://github.com/castorgit/AGENTS-course/blob/main/HW1/Hello_World_Ollama.ipynb
```

11 Hardware Requirements and Troubleshooting

Machine	Recommended Model	Expected Speed
Laptop, no GPU (8GB RAM)	`qwen2.5:3b`	∼5–10 tok/s
Laptop, no GPU (16GB RAM)	`qwen2.5:7b`	∼8–15 tok/s
Laptop with GPU (8GB VRAM)	`qwen2.5:7b`	∼30–60 tok/s
Workstation (24GB+ VRAM)	`qwen2.5:14b`	∼40–80 tok/s

Common issues:

- **"connection refused" error:** Ollama is not running. Start it with `ollama serve` in a separate terminal.
- **Very slow responses:** You are running on CPU only. This is normal — switch to a smaller model (3b) or be patient.
- **Out of memory crash:** Your machine does not have enough RAM. Use `qwen2.5:3b` or `qwen2.5:0.5b`.
- **Model not found:** Run `ollama pull qwen2.5:7b` first, then restart your Python script.
- **Windows port conflict:** If port `11434` is blocked, set `OLLAMA_HOST=0.0.0.0:11435` and update your API calls accordingly.

12 Deliverables and Submission

You must submit a single **zip file** containing:

- **Python script(s):** Your complete application (`app.py` and any helper modules). Must run with `streamlit run app.py` after installing dependencies.
- **Requirements file** (`requirements.txt`): Generated with `pip freeze > requirements.txt`.
- **README** (`README.md`): A short file (10–20 lines) explaining how to install dependencies and run the application.
- **Screenshot** (`screenshot.png`): A screenshot of your running chat interface showing a multi-turn conversation of at least 4 exchanges.

- **Short reflection** (`reflection.pdf`, max 1 page): Answer the following questions:
  1. What are the privacy advantages of this approach compared to using the ChatGPT API?
  2. What are the main limitations of running a model locally?
  3. In what professional or personal scenario would you use a local LLM?

References

[1] Gradio Team. *Gradio Documentation*. https://www.gradio.app/docs. 2024.

[2] Joao Moura. *CrewAI: Orchestrating Role-Playing, Collaborative AI Agents*. Version 0.30.0. Accessed: April 2026. 2024. URL: https://github.com/joaomoura/crewai.

[3] Ollama Team. *Ollama Documentation and Model Library*. https://ollama.com. Accessed: 2026-03-02. 2024.

[4] Ollama Team. *Ollama Python SDK*. https://github.com/ollama/ollama-python. 2024.

[5] Qwen Team. *Qwen 2.5 Model Card and Technical Report*. https://huggingface.co/Qwen. Hugging Face Repository. 2024.

[6] R. Shao et al. "Privacy-Preserving LLM Inference: A Survey". In: *arXiv preprint arXiv:2405.01123* (2024). Survey on local inference and on-device AI. URL: https://arxiv.org/abs/2405.01123.

[7] Streamlit Team. *Streamlit Documentation*. https://docs.streamlit.io. 2024.